



THE REAL CHEAT SHEET + PLAYBOOK

CLAUDE CODE vs CODEX

I gave both the same brain and ran them on real client work — not benchmarks. Page 1 is who won. The rest is how to make **Claude Code edit your videos** the same way I do.

3

CLAUDE CODE WINS

1

TIE

1

CODEX WINS

The Verdict — round by round

Graphic Design

logos, image gen, creative

Claude



40%

Codex



90%

CODEX 🏆

Video Editing

clean, watchable cuts

Claude



85%

Codex



85%

TIE 🤝

Automation

n8n · GoHighLevel · MCP

Claude



92%

Codex



68%

CLAUDE 🏆

Development

systems, full builds

Claude



94%

Codex



72%

CLAUDE 🏆

One rule to remember

Use **Codex** for quick creative passes (design, image models). Use **Claude Code** for systems, automation, dev — and for driving a real video pipeline. Don't pick one; route the task to whoever's strongest.

Tested on real work, not benchmarks.

The next 4 pages = the exact system. Built by [@SkynetLabs](#).



— The big idea

Claude Code isn't a video editor with buttons. It's an **AI engineer that drives the tools** — mainly **ffmpeg** — from plain-English instructions. You give it footage + a brief; it writes and runs the commands, checks the frames, and fixes its own mistakes. That's why it ties a dedicated editor on output and beats it on **repeatability**: once it works, it runs the same way every time.

— What you need (one-time setup)

```
# 1. Claude Code (the CLI) npm install -g
@anthropic-ai/claude-code

# 2. ffmpeg — the actual editing engine
winget install ffmpeg
# Windows brew install
ffmpeg # macOS

# 3. (optional) Whisper for auto-captions, ElevenLabs key for AI
voiceover
pip install openai-whisper
```

— Set up the project folder

Claude Code works best with a clean, predictable folder. Drop this structure in and it always knows where things are:

```
my-edit/ └─ raw/
# original clips go here └─
assets/
# logos, music, fonts, overlay PNGs └─
out/
# rendered exports └─
captions.ass
# subtitle file (auto-made from whisper) └─
CLAUDE.md
# your editing rules — read every session
```

The secret: CLAUDE.md is your editor's brain

Put your **style rules** in a **CLAUDE.md** file in the project. Claude reads it automatically every time. This is how you "train" it — facts that are always true (export specs, brand colours, your hooks). Page 4

has a ready-to-paste version.

Setup once → edit forever.

Next page: the 7-step editing pipeline Claude runs.

page 2 / 5



— The workflow — what to ask, in order

1**Brief the edit**

Tell it the goal, platform, and vibe. "Cut raw/talk.mp4 into a 9:16 reel for TikTok, punchy, captions on, 30-45s."

2**Transcribe & find the cuts**

Whisper makes a transcript + `captions.ass`. Claude reads it to drop dead air and find the strongest 30 seconds.

3**Frame & crop (face-safe)**

Reframe to 9:16. Keep the speaker off-centre so overlays have room. Never crop through the face — this is the #1 rule.

4**Grade the colour**

One pass: lift contrast + saturation, warm the highlights. Makes phone footage look graded, not flat.

5**Overlays & titles**

Burn captions, hook card, scorecards, tool chips (n8n / GHL / MCP). Use PNG overlays, not alpha video (see gotchas).

6**Audio: voiceover + ducked music**

Optional ElevenLabs VO. Sidechain-duck the music under the voice so every word is clear.

7**Render + QA frame-by-frame**

Export, then have Claude pull sample frames and check: face never cut, captions in sync, no black frames. Fix, re-render.

Why this beats "do it all in one prompt"

Each step is checkable. Claude verifies its own output between steps and self-corrects — that's the part a human editor can't do at 2am, and it's why the result is **consistent** across 20 videos, not just one lucky render.

Next: the master prompt + skill that runs all of it.



— 1 · The master prompt (paste into Claude Code)

► PASTE THIS, SWAP THE BRACKETS

You are my video editor. Edit **raw/[CLIP].mp4** into a **[9:16 / 16:9]** [TikTok/YouTube] video, target **[30-45s]**.

Pipeline: transcribe → cut dead air → reframe face-safe → colour grade → burn captions + a hook card → optional VO + ducked music → render → pull 6 sample frames and QA (face never cropped, captions synced, no black frames). Fix issues and re-render.

Use ffmpeg. Read CLAUDE.md for my style rules. Show me each command before the final render. Output to **out/**.

— 2 · Your CLAUDE.md style rules (the "training")

```
# Video Editing Rules
Export: 1080x1920 (9:16) or 1920x1080 (16:9),
30fps, H.264, yuv420p. Encoder: try h264_nvenc;
if it errors, fall back to libx264 -crf 18.
Face rule: never crop through the speaker's
face. Keep head in frame. Captions: bold,
bottom third, high contrast, 2-4 words per line.
Brand: accent = orange #ef7a52 + green #34d399.
Font = Archivo Black for titles. Audio: music
ducks ~12dB under voice. Normalise to -14 LUFS.
Overlays: PNG only (alpha .mov fails on this
machine). QA: before "done", pull sample frames
and verify face + captions + no black frames.
```

— 3 · Make it a reusable Skill (so you never re-type)

Save the prompt + rules as a Skill and trigger it by name forever. Drop this in
~/claude/skills/video-edit/SKILL.md:

```
--- name: video-edit
description:
Edit a raw clip into a platform-ready reel — cut,
reframe, grade, caption, voiceover, render + QA. Trigger on
"edit this video", "make a reel", "/video-edit".
---
# then paste the 7-step pipeline + CLAUDE.md rules here
```

Now it's one command

Type **/video-edit** + drop a clip. Claude runs the whole pipeline with your rules baked in. That's a \$0, repeatable editing bay that never forgets your style.

Prompt → rules → skill.

Last page: ffmpeg recipes + the traps that waste hours.

page 4 / 5



— ffmpeg recipes — copy/paste

```
# Vertical 9:16 crop, centred on subject
ffmpeg -i raw/in.mp4 -vf
"scale=-1:1920,crop=1080:1920" out/vert.mp4

# Quick cinematic colour grade
ffmpeg -i in.mp4 -vf
"eq=contrast=1.08:saturation=1.15:brightness=0.02"
graded.mp4

# Burn captions into the video
ffmpeg -i in.mp4 -vf
"subtitles=captions.ass" captioned.mp4

# Duck music under the voiceover (sidechain)
ffmpeg -i music.mp3 -i vo.wav -filter_complex \
"[0:a][1:a]sidechaincompress=threshold=0.03:ratio=8[a]"
-map "[a]" mix.wav

# Trim dead air (keep 4.0s-32.5s)
ffmpeg -ss 4.0 -to 32.5 -i in.mp4 -c copy clip.mp4

# Final render — NVENC with safe libx264 fallback
ffmpeg -i edit.mp4 -c:v h264_nvenc -b:v 10M -pix_fmt yuv420p final.mp4
\
# if NVENC errors: -c:v libx264 -crf 18
-preset slow
```

— The traps that waste hours

TRAP	WHAT HAPPENS	FIX
NVENC fails	GPU encoder errors / green frames on some machines	Fall back to <code>libx264 -crf 18</code>
Alpha .mov overlay	Transparent video silently drops — overlay vanishes	Export overlay as <code>PNG</code> and overlay that
Face cropped	9:16 crop cuts through the speaker's head	Off-centre the subject; QA frames before done
Captions drift	Subtitles slip out of sync after a trim	Caption <code>after</code> cutting, not before

— Which AI to open — by job

Reach for Claude Code

- › Driving the full video pipeline
- › Automations: n8n, GoHighLevel, MCP
- › Multi-file builds & refactors
- › Anything you'll ship to a client

Reach for Codex

- › Graphic design & image generation
- › Fast one-off creative drafts
- › Quick scripts & snippets
- › Speed over depth

Want builds & edits like this done for you?

AI agents, automations, sites & reels — DM [@SkynetLabs](#) · comment **STACK** for more.

skynetlabs

v2 · 2026 · page 5 /

5